

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CÔNG VIỆC TUẦN

TÌM HIỂU KIẾN TRÚC CASE STUDY MAJOR TRAUMA
MANAGEMENT
TRÊN HYPERMEDIA-BASED WEB OF DIGITAL TWINS (WoDT)
FRAMEWORK

Họ và tên: Nguyễn Khánh Việt

Mã sinh viên: 23020171

Lớp: K68I-IT3

Tuần báo cáo: Tuần 6

Giảng viên hướng dẫn: PGS. TS. Đặng Đức Hạnh

Hà Nội, năm 2026

1. Nội dung công việc trong tuần

Tiếp nối kế hoạch tuần 5 (tìm case study có thể mô hình hoá bằng BPMN/goal model biến thể cao để áp dụng cho bài toán Digital Twin), trong tuần thứ sáu em đã lựa chọn và tìm hiểu case study **Major Trauma Management**, được triển khai trong repository <https://github.com/Web-of-Digital-Twins/major-trauma-management-case-study> dựa trên *Hypermedia-based WoDT Framework* (bám theo mô tả lý thuyết trong bài báo *Web of Digital Twins* của A. Ricci và cộng sự). Case study này mô phỏng hai giai đoạn đầu của quy trình cấp cứu chấn thương: *Emergency Call Management* và *Pre-Hospital Management*. Nội dung chính đã tìm hiểu gồm:

- **Kiến trúc tổng thể:** hệ sinh thái Digital Twin không đồng nhất, được ghép nối qua nền tảng *WoDT Platform* đóng vai trò *knowledge graph (KG)* trung tâm; mỗi DT được triển khai trên một trong ba framework khác nhau và giao tiếp với nhau thông qua Web of Things (WoT) Thing Description (TD)/Digital Twin Description (DTD) theo nguyên lý hypermedia (điều hướng hệ thống qua các affordance được nhúng trong chính mô tả của DT, tương tự HATEOAS);
- **Các Digital Twin tham gia và framework triển khai tương ứng:** *White Label Digital Twins Framework (WLDT)* triển khai Mission DT, Ambulance DT, Rescuer DT; *Azure Digital Twins* triển khai Patient DT và Ongoing(Trauma) DT; *Eclipse Ditto* triển khai Healthcare User DT – cho thấy kiến trúc chủ động chấp nhận tính không đồng nhất về công nghệ, dùng lớp *adapter* (WoDT adapter, Azure Function adapter) để đồng bộ trạng thái DT vào KG chung;
- **Mô hình dữ liệu ngữ nghĩa (semantic/ontology):** các thực thể và quan hệ trong KG được biểu diễn theo ontology FHIR (fhir:Location, fhir:Practitioner, fhir:qualification, fhir:status...) kết hợp ontology riêng cho bệnh nhân (onto:patient) và từ vựng WoT/hypermedia (wot:hasForm, hypermedia:hasOperationType, hypermedia:hasTarget) để mô tả cả *trạng thái nghiệp vụ lẫn giao diện tương tác* của từng DT trong cùng một đồ thị tri thức;
- **Luồng nghiệp vụ (business logic) dựa trên truy vấn SPARQL trên KG thay vì luồng BPMN tường minh:** khi có cuộc gọi cấp cứu, một *Mission DT* được tạo; một *Mission Agent* quan sát KG và thực hiện truy vấn SPARQL để tìm xe cứu thương đang rảnh (status = "inactive") và nhân viên cứu hộ có chuyên môn *Paramedic* chưa được gán cho ca trực nào (dùng FILTER NOT EXISTS), sau đó gán nguồn lực bằng cách tạo quan hệ (relationship) giữa các DT trong KG;

- **Điều khiển thiết bị qua DTD:** Mission Agent lấy DTD của Ambulance DT và gọi hành động `SetDestinationCommand` để thiết lập điểm đến – minh họa cách một DT vừa biểu diễn trạng thái, vừa lộ ra *affordance* hành động có thể gọi từ bên ngoài;
- **Liên kết DT theo thời gian thực và khám phá dịch vụ:** khi xe cứu thương tới hiện trường, *Patient DT* được tạo và liên kết với Mission DT; mã số bệnh nhân được dùng để tra cứu *Healthcare User DT* tương ứng qua *Platform DT Discovery Service*; đội cấp cứu tại bệnh viện chỉ cần một truy vấn SPARQL trên URI của Mission DT để lấy ra endpoint quan sát (observe) theo giao thức WebSocket của cả Ambulance DT lẫn Patient DT, thay vì phải biết trước địa chỉ của từng thiết bị;
- **Các agent/ứng dụng nghiệp vụ trên nền KG:** ngoài Mission Agent, hệ thống còn có *Documentation Agent* (tự sinh tài liệu quy trình cấp cứu từ việc quan sát KG) và *CEU Dashboard* (truy vấn SPARQL để theo dõi các mission đang on-hold/in-progress) – cho thấy cùng một KG có thể phục vụ nhiều goal/ng nghiệp vụ khác nhau chỉ bằng các truy vấn khác nhau, không cần thay đổi các DT bên dưới;
- **Triển khai hạ tầng:** toàn bộ hệ thống được đóng gói bằng Docker Compose, với các cổng dịch vụ cố định cho WoDT Platform và từng DT, cùng các biến môi trường cấu hình kết nối tới Azure Digital Twins, Eclipse Ditto và Azure SignalR.

2. Cách thức thực hiện

1. Đọc README và cấu trúc thư mục của repository (`azure-dts`, `ditto-dts`, `ditto-initializer`, `wldt-dts`, `docker-compose.yml`) để xác định DT nào được triển khai trên framework nào và cách chúng được khởi chạy cùng nhau;
2. Đọc lại phần liên quan trong bài báo gốc *Web of Digital Twins* (Ricci et al.) để hiểu bối cảnh lý thuyết của case study – khái niệm DTD, WoDT Platform, Digital Twin Engine – trước khi đối chiếu với phần triển khai thực tế trong repository;
3. Vẽ lại sơ đồ luồng nghiệp vụ "*A possible journey*" mô tả trong README (từ nhận cuộc gọi khẩn cấp → tạo Mission DT → gán nguồn lực → điều khiển xe cứu thương → tạo Patient DT → liên kết Healthcare User DT → cấp quyền quan sát cho đội cấp cứu → tạo Ongoing DT) để dễ đối chiếu với các khái niệm goal/VP đã học ở tuần 5;
4. Phân tích các truy vấn SPARQL mẫu trong README (tìm nguồn lực, lấy endpoint quan sát, theo dõi mission đang hoạt động) để hiểu cách nghiệp vụ được biểu diễn dưới dạng truy vấn trên đồ thị tri thức thay vì một quy trình BPMN/goal model tường minh;

5. So sánh sơ bộ với hướng tiếp cận goal model/BPEL đã tìm hiểu tuần 5: cùng có khái niệm ”điểm quyết định/biến thể” trong vận hành (ví dụ lựa chọn ambulance/rescuer nào), nhưng case study này hiện thực hoá biến thể đó bằng truy vấn động trên KG (*data-driven*, xử lý tại runtime) thay vì bằng các *variation point* được xác định trước ở mức thiết kế và sinh mã BPEL;
6. Ghi chú các thành phần có thể tái sử dụng khi thử triển khai ở tuần 7: yêu cầu hạ tầng (Docker, Eclipse Ditto, Azure Digital Twins, Azure Function adapter, thư viện wldt-wodt-adapter), các biến môi trường cần cấu hình trong file .env, và các cổng dịch vụ tương ứng với từng DT.

3. Kết quả thu được và bài học rút ra

3.1. Kết quả thu được

- Nắm được kiến trúc tổng thể của một hệ sinh thái Digital Twin thực tế gồm nhiều framework khác nhau (WLDT, Azure Digital Twins, Eclipse Ditto), được hợp nhất thông qua một *knowledge graph* trung tâm và các adapter theo chuẩn hypermedia;
- Hiểu được vai trò của DTD/TD như là ”giao diện tương tác” hợp nhất của DT (vừa mô tả thuộc tính/trạng thái, vừa lộ ra hành động và endpoint quan sát), giúp các agent/ứng dụng nghiệp vụ không cần biết trước địa chỉ hay giao thức cụ thể của từng thiết bị;
- Hiểu cách một quy trình nghiệp vụ có tính thời gian thực và nhiều biến thể (chọn ambulance, chọn rescuer, liên kết bệnh nhân) có thể được hiện thực hoàn toàn bằng truy vấn SPARQL động trên KG, thay vì bằng một mô hình quy trình tĩnh được biên dịch trước;
- Xác định được danh sách yêu cầu hạ tầng và các bước cần chuẩn bị (Docker, Eclipse Ditto, Azure Digital Twins, biến môi trường) để có thể thử triển khai case study trong tuần 7.

3.2. Bài học rút ra

- Đối với hệ Digital Twin có tính không đồng nhất công nghệ cao (nhiều framework khác nhau), việc chọn một mô hình dữ liệu chung dạng *knowledge graph* kèm ontology (ở đây là FHIR) là yếu tố then chốt để các thành phần có thể liên vận hành mà không cần biết chi tiết triển khai của nhau;

- Cách case study này xử lý "biến thể vận hành" bằng truy vấn KG tại runtime là một hướng bổ sung đáng chú ý so với hướng goal model/VP tĩnh đã tìm hiểu tuần 5: hai hướng có thể kết hợp – goal model dùng để thiết kế/phân tích các phương án ở mức yêu cầu, còn KG + SPARQL dùng để hiện thực hoá việc lựa chọn phương án đó một cách linh hoạt tại runtime;
- Nguyên lý hypermedia (điều hướng hệ thống qua các affordance nhúng trong DTD) là một gợi ý tốt cho việc thiết kế phần "giao diện tương tác" của tri thức Digital Twin trong đề tài, giúp tách bạch giữa mô hình tri thức (KG/ontology) và cách các tác nhân bên ngoài tương tác với DT.

4. Kế hoạch công việc cho tuần sau

1. Chuẩn bị hạ tầng cần thiết (Docker, Docker Compose, Eclipse Ditto) và, nếu điều kiện cho phép, một instance Azure Digital Twins cùng Azure Function adapter để thử chạy case study Major Trauma Management theo hướng dẫn trong README;
2. Cấu hình file .env với các biến kết nối (AZURE_*, DITTO_*, SIGNALR_*) và chạy thử `docker-compose up` để khởi động toàn bộ hệ sinh thái DT (Healthcare User DT, Mission DT, Ambulance DT, Rescuer DT, Patient DT, Ongoing DT);
3. Thử thực hiện lại kịch bản "*A possible journey*": đăng ký thủ công Ambulance/Rescuer/Healthcare User DT, mô phỏng một cuộc gọi cấp cứu, quan sát Mission Agent thực hiện truy vấn SPARQL và gán nguồn lực, theo dõi việc tạo Patient DT và Ongoing DT;
4. Ghi nhận lại các vướng mắc khi triển khai (nếu có) và đối chiếu kết quả thực nghiệm với phần kiến trúc đã tìm hiểu ở tuần 6, làm cơ sở để tuần sau đó bắt đầu phác thảo việc ánh xạ case study này sang hướng gStar (Event-B/LTL) của đề tài.

5. Tự đánh giá tiến độ

Trong tuần này, em đã hoàn thành việc tìm hiểu kiến trúc và cơ chế vận hành của case study Major Trauma Management trên Hypermedia-based WoDT Framework, bao gồm: kiến trúc đa framework hợp nhất qua knowledge graph, mô hình dữ liệu ngữ nghĩa theo ontology FHIR, cơ chế điều khiển/quan sát DT qua DTD, và cách nghiệp vụ được hiện thực bằng truy vấn SPARQL động. Kết hợp với nền tảng lý thuyết về goal model/variation point ở tuần 5, em đã có đủ hiểu biết cần thiết để chuyển sang giai đoạn triển khai và chạy thử case study ở tuần 7. Tiến độ bám sát kế hoạch chung của đợt thực tập.

References

- [1] Web-of-Digital-Twins, *Major Trauma Management Case Study – Implementing the Major Trauma Management case study with the Hypermedia-based WoDT Framework*, GitHub repository, <https://github.com/Web-of-Digital-Twins/major-trauma-management-case-study>.
- [2] A. Ricci, A. Croatti, S. Mariani, S. Montagna, M. Picone, *Web of Digital Twins*, ACM Transactions on Internet Technology, 2022.
- [3] A. Lapouchnian, Y. Yu, J. Mylopoulos, *Requirements-Driven Design and Configuration Management of Business Processes*, Proc. Business Process Management (BPM), 2007.
- [4] iStar 2.0 Language Guide, *The iStar 2.0 Core Language*.
- [5] La Trịnh Hoàng Việt, *Nghiên cứu phát triển khung ngữ nghĩa hình thức cho các mô hình yêu cầu định hướng mục tiêu*, Luận văn Thạc sĩ, Trường Đại học Công nghệ – ĐHQGHN, 2026.